Project Report

On

Performance Analysis and Optimization of the N-Body Problem Using Parallel Programming Paradigms

Submitted

in partial fulfillment

of the requirements for the course

Advanced Certification Course – High Performance Computing (ACC-HPC)

Guided By :-

Mr. Jayesh Deep Dubey

(Scientist B)

Submitted By :-

Sagar Kumar Chaubey	250880750023
Aaditya Kumar	250880750001
Payal Kumari	250880750014
Anuj Kumar	250880750004

Centre for Development of Advanced Computing (C-DAC) Patna

Academic Year: 2025–2026

Abstract

The N-Body problem is a fundamental computational kernel in scientific computing, widely used to study performance behavior in High Performance Computing (HPC) systems due to its quadratic computational complexity and intensive data movement. This project presents a comprehensive performance engineering study of the classical N-Body simulation implemented across multiple execution paradigms, including Serial, OpenMP, MPI, Hybrid MPI + OpenMP, and CUDA.

Starting from baseline implementations, the project systematically applies layered optimization techniques such as data layout restructuring, SIMD vectorization using AVX-512, cache blocking, NUMA-aware thread placement, communication optimization in MPI, and GPU shared-memory tiling. For each paradigm, both baseline and highly optimized "Ultra" versions are developed and evaluated on the Paramutkarsh HPC cluster under controlled experimental conditions.

Extensive profiling and performance analysis reveal significant improvements in computational throughput, scalability, and resource utilization. The optimized Hybrid CPU implementation achieves over 1.4 TFLOPs, while the CUDA Ultra implementation surpasses 3.8 TFLOPs, establishing the global performance ceiling of the study. The results demonstrate that effective performance engineering requires a disciplined approach combining algorithmic optimization, architectural awareness, and parallel execution.

This work highlights key principles of modern HPC design and provides a practical framework for developing high-performance scientific applications on contemporary CPU–GPU heterogeneous systems.

Table of Contents

Abstract.....1

Table of Contents..... 2

Chapter 1 — Introduction..... 1

1.1 Background and Motivation.....1

1.2 Problem Statement..... 2

1.3 Project Objectives..... 2

1.4 Scope of Work.....2

Chapter 2 — Literature Review.....3

2.1 Evolution of N-Body Computing.....3

2.2 Parallel Programming Paradigms.....3

2.3 Optimization Techniques in HPC.....3

Chapter 3 — Methodology and Implementation.....4

3.1 Overall System Architecture..... 4

3.2 Numerical Model and Simulation Loop.....4

3.3 Baseline Implementation Strategy..... 5

3.4 Experimental Platform and System Configuration..... 5

3.5 Serial Baseline Implementation..... 8

3.6 Serial Ultra Implementation.....12

3.7 OpenMP Baseline Implementation..... 17

3.8 OpenMP Ultra Implementation.....19

3.9 MPI Baseline Implementation.....22

3.10 MPI Ultra Implementation..... 25

3.11 Hybrid MPI + OpenMP Baseline Implementation.....28

3.12 Hybrid MPI + OpenMP Ultra Implementation..... 30

3.13 CUDA Baseline Implementation..... 32

3.14 CUDA Ultra Implementation.....34

Chapter 4 — Performance Engineering.....36

4.1 Serial Performance Analysis.....36

4.2 OpenMP Performance Analysis.....37

4.3 MPI Performance Analysis.....38

4.4 Hybrid MPI + OpenMP Performance Analysis.....39

4.5 Cross-Paradigm Performance Comparison.....40

4.6 Roofline & Scalability Discussion.....40

4.7 GPU Acceleration Analysis (CUDA).....41

4.8 Final Cross-Platform Performance Summary.....43

4.9 Performance Analysis Graphs.....43

Chapter 5 — Experimental Results and Discussion.....47

5.1 Experimental Methodology.....47

5.2 Correctness and Stability Verification.....47

5.3 Performance Trends.....47

5.4 Bottleneck Evolution.....48

5.5 Key Engineering Insights.....48

5.6 Summary of Findings.....48

Chapter 6 — Conclusion and Future Work.....49

6.1 Conclusion.....49

6.2 Key Contributions.....49

6.3 Lessons Learned.....50

6.4 Future Work.....50

6.5 Final Remark.....50

References.....51

Chapter 1 — Introduction

1.1 Background and Motivation

Modern computational science is no longer limited by correctness alone; it is bounded by performance, scalability, and efficiency. Contemporary processors expose parallelism at multiple layers:

Layer	Source of Parallelism
Instruction Level	SIMD, pipelining
Thread Level	Multi-core CPUs
Node Level	NUMA domains
Cluster Level	Distributed nodes via interconnect
Accelerator Level	GPUs, many-core devices

The N-Body problem sits at the intersection of all these layers.

Mathematical Model

For each body i:

$$\vec{F}_i = \sum_{j \neq i} G \cdot \frac{m_i m_j}{(||\vec{r}_j - \vec{r}_i||^2 + \epsilon)^{3/2}} \cdot (\vec{r}_j - \vec{r}_i)$$

Velocity and position updates:

$$\vec{v}_i(t + \Delta t) = \vec{v}_i(t) + \frac{\vec{F}_i}{m_i} \cdot \Delta t$$

$$\vec{r}_i(t + \Delta t) = \vec{r}_i(t) + \vec{v}_i(t + \Delta t) \cdot \Delta t$$

Computational Characteristics

Property	Behavior
Time Complexity	$O(N^2)$

Arithmetic Intensity	Very High
Memory Access	Irregular, bandwidth-heavy
Parallelism	Massive and natural
Scaling Sensitivity	Extreme

This makes N-Body a perfect performance microscope for HPC research.

1.2 Problem Statement

Given N interacting bodies, simulate their motion under Newtonian gravity for T time steps on modern heterogeneous HPC systems while:

- Preserving numerical stability
- Maximizing hardware utilization
- Maintaining scalability across architectures

1.3 Project Objectives

ID	Objective
O1	Implement N-Body using 5 programming paradigms
O2	Design both baseline and ultra-optimized versions
O3	Apply multi-layer performance optimization
O4	Evaluate scaling behavior across CPU & GPU
O5	Analyze performance using profiling & benchmarking

1.4 Scope of Work

Layer	Covered
Algorithm Design	✔
Parallel Programming	✔
Optimization	✔
Distributed Communication	✔
GPU Acceleration	✔
Performance Modeling	✔

Chapter 2 — Literature Review

2.1 Evolution of N-Body Computing

Era	Dominant Technique
1970s–90s	Direct $O(N^2)$ CPU computation
1990s–2000s	Barnes–Hut, FMM
2005–2015	MPI clusters
2015–Present	Hybrid CPU–GPU systems

This project intentionally uses direct $O(N^2)$ to expose raw architectural behavior.

2.2 Parallel Programming Paradigms

Paradigm	Strength	Limitation
OpenMP	Simple shared-memory scaling	NUMA & bandwidth bottlenecks
MPI	Distributed scalability	Communication overhead
Hybrid	Balanced node utilization	Increased complexity
CUDA	Massive throughput	Memory hierarchy complexity

2.3 Optimization Techniques in HPC

Technique	Benefit
SoA Layout	Improves cache & SIMD efficiency
SIMD Vectorization	Increases FLOP throughput
Cache Blocking	Reduces memory stalls
NUMA Pinning	Eliminates cross-domain latency
MPI Topology Tuning	Reduces network congestion
GPU Shared Memory	Minimizes global memory traffic

Chapter 3 — Methodology and Implementation

3.1 Overall System Architecture

The N-Body simulation framework developed in this project is organized as a layered performance study environment. Each computational paradigm is implemented as an independent module following the same physical model and numerical update scheme, enabling direct and fair comparison across architectures.

Implementation Variants

Category	Variant	Purpose
CPU	Serial	Baseline reference
CPU	OpenMP	Shared-memory parallelism
CPU	MPI	Distributed-memory parallelism
CPU	Hybrid (MPI + OpenMP)	Node-level + cluster-level parallelism
GPU	CUDA	Accelerator computing

Each variant contains two layers:

- **Baseline**: Correctness, clarity, structural design
- **Ultra**: Aggressive optimization and performance tuning

All variants share identical input formats, simulation parameters, and output specifications.

3.2 Numerical Model and Simulation Loop

Each time step performs three phases:

Phase	Description
Force Computation	Compute all-pairs gravitational forces
Integration	Update velocities
Position Update	Advance positions

Discrete Update Formulation

$$\vec{F}_i = \sum_{j \neq i} G \cdot \frac{m_i m_j}{(||\vec{r}_j - \vec{r}_i||^2 + \epsilon)^{3/2}} \cdot (\vec{r}_j - \vec{r}_i)$$

$$\vec{v}_i \leftarrow \vec{v}_i + \frac{\vec{F}_i}{m_i} \cdot \Delta t$$

$$\vec{r}_i \leftarrow \vec{r}_i + \vec{v}_i \cdot \Delta t$$

3.3 Baseline Implementation Strategy

Before optimization, each CPU variant is designed with the following priorities:

Design Principle	Rationale
Correctness	Ensures physical validity
Readability	Facilitates analysis
Deterministic Behavior	Enables fair benchmarking
Portability	Works across architectures

3.4 Experimental Platform and System Configuration

This section describes the complete hardware and software environment used for the implementation, execution, and performance evaluation of the N-Body simulation. All experiments were conducted on the **Paramutkarsh High Performance Computing (HPC) cluster at C-DAC** under controlled conditions to ensure reproducibility and fairness of results.

3.4.1 Operating System and Software Environment

Component	Specification
Operating System	CentOS Linux 7 (Core)
OS Type	64-bit
Kernel Version	3.10.0-1160.el7.x86_64

Distribution Family	RHEL-compatible
Compiler	GCC (g++)
MPI Library	OpenMPI / MPICH
OpenMP Runtime	GCC OpenMP
CUDA Toolkit	NVIDIA CUDA 11.0
Job Scheduler	SLURM

This environment provides stable long-term support and compatibility with modern HPC libraries and toolchains.

3.4.2 Login Node Configuration

Login nodes are used exclusively for compilation, job submission, and preprocessing. No performance experiments are executed on login nodes.

Property	**Value**
Hostname	login03.cdac.in
Processor	Intel Xeon Gold 6248
Sockets	2
Cores per Socket	20
Total Cores	40
Threads per Core	1
Base Frequency	2.50 GHz
Max Turbo Frequency	3.90 GHz
NUMA Nodes	2

Cache Hierarchy:

Level	**Size**
L1 Data	32 KB / core
L1 Instruction	32 KB / core
L2	1 MB / core
L3	28 MB shared

3.4.3 CPU Compute Node Configuration (Partition: cpu)

All Serial, OpenMP, MPI, and Hybrid experiments were performed on CPU compute nodes.

Property	**Value**
Compute Node	cn066

Processor	Intel Xeon Platinum 8268
Sockets	2
Cores per Socket	24
Total Cores	48
Threads per Core	1
Base Frequency	2.90 GHz
Max Turbo Frequency	3.90 GHz
NUMA Nodes	2

NUMA Topology

NUMA Node	Core Range
Node 0	0 – 23
Node 1	24 – 47

This dual-socket NUMA architecture enables evaluation of memory locality, thread placement, and hybrid execution behavior.

3.4.4 Memory Configuration

Node Type	Total RAM	NUMA Distribution
Login Node	~376 GB	Node0: ~187 GB, Node1: ~189 GB
Compute Node	~188 GB	Node0: ~93 GB, Node1: ~94 GB

Large memory capacity prevents paging and supports large-scale N-Body simulations.

3.4.5 GPU Node Configuration (Partition: gpu)

Property	Value
GPU Node	gpu005
Host CPU	Intel Xeon Gold 6248
Total CPU Cores	40
NUMA Nodes	2

GPU Model	NVIDIA Tesla V100-SXM2
GPU Memory	16 GB HBM2
CUDA Cores	5120
Driver Version	450.51.05
CUDA Version	11.0
ECC Mode	Enabled

The Tesla V100 provides high double-precision throughput suitable for scientific N-Body workloads.

3.4.6 Interconnect and Networking

Property	Specification
Interconnect	Mellanox ConnectX-6 InfiniBand
Link Speed	100 Gbps
MTU	4096 bytes
Transport	InfiniBand
NUMA Affinity	NUMA-aware network adapters

3.4.7 Relevance to Performance Evaluation

The heterogeneous architecture of Paramutkarsh enables systematic evaluation of:

- Shared-memory parallelism (OpenMP)
- Distributed-memory parallelism (MPI)
- Hybrid parallelism (MPI + OpenMP)
- Accelerator-based computing (CUDA)

Using consistent hardware configurations ensures that all performance results reflect true algorithmic and architectural behavior rather than environmental noise.

3.5 Serial Baseline Implementation

The serial implementation serves as the foundational reference for all subsequent parallel and optimized versions of the N-Body simulation. It establishes the physical

model, numerical update scheme, execution structure, and performance measurement methodology in a form that is easy to verify for correctness and reproducibility.

This baseline version is intentionally designed without any form of parallelism or hardware-specific optimization in order to expose the inherent computational characteristics of the N-Body problem.

3.5.1 Program Structure Overview

The serial program is organized into the following major components:

1. **Command-line configuration and mode selection**
2. **System initialization and body generation**
3. **Main simulation loop**
4. **Physics computation kernel**
5. **Trajectory output handling (visual mode)**
6. **High-resolution performance measurement and reporting**

Each component is executed in a strictly deterministic order to ensure repeatable experimental results.

3.5.2 Initialization Phase

Randomized Body Generation

At program startup, the simulation allocates a vector of Body structures, where each body contains:

- Position: *(x, y, z)*
- Velocity: *(vx, vy, vz)*
- Mass

A fixed random seed (SEED = 1234) is used to ensure identical initial conditions across different program executions and across different computational variants.

The initialization procedure assigns:

- Positions uniformly in the range **[-1, 1]**
- Velocities in the range **[-0.5, 0.5]**
- Mass in the range **[0.1, 1.0]**

This controlled initialization is critical for fair cross-paradigm performance comparison.

3.5.3 Physics Computation Kernel

At each simulation step, the program evaluates the gravitational interaction between all pairs of bodies, following the direct $O(N^2)$ formulation:

For each body **i**:

1. Accumulate gravitational forces from all bodies **j ≠ i**
2. Apply Newton's law of gravitation with softening:

$$F = \frac{G \cdot m_i \cdot m_j}{(r^2 + \epsilon)^{3/2}}$$

3. Compute acceleration:

$$a_i = \frac{F_i}{m_i}$$

4. Update velocity:

$$v_i(t + \Delta t) = v_i(t) + a_i \cdot \Delta t$$

5. Update position:

$$x_i(t + \Delta t) = x_i(t) + v_i(t + \Delta t) \cdot \Delta t$$

Temporary force arrays *(fx, fy, fz)* are used to decouple force accumulation from integration, improving numerical clarity.

3.5.4 Execution Flow Diagram of Serial Program

The following execution flow captures **every operational stage** of the program, including argument handling, I/O, timing, computation, and termination:

```
Program Start
  |-- Parse Command-Line Arguments
  |      - n_bodies
  |      - n_steps
  |      - mode (bench / visual)
  |      - save_interval
  |-- Configure Output Mode
  |      - Benchmark Mode → no file output
  |      - Visual Mode    → enable trajectory logging
  |-- Allocate Body Array
  |-- Initialize Bodies (fixed SEED)
```

```
|       - Assign positions
|       - Assign velocities
|       - Assign masses
|-- If Visual Mode:
|       - Open CSV output file
|       - Write header row
|-- Start High-Resolution Timer
|-- For step = 0 → n_steps - 1:
|       |-- If Visual Mode AND step % save_interval == 0:
|       |       |-- For each body i:
|       |               Write:
|       |               step, id, position, velocity, mass
|       |-- Compute Physics
|               |-- For each body i:
|                       |-- Reset force accumulators
|                       |-- For each body j ≠ i:
|                               |-- Compute distance vector
|                               |-- Apply softening
|                               |-- Compute force contribution
|                               |-- Accumulate fx, fy, fz
|               |-- For each body i:
|                       |-- Compute acceleration
|                       |-- Update velocity
|                       |-- Update position
|-- End For Loop
|-- Stop High-Resolution Timer
|-- If Visual Mode:
|       - Close output file
|-- Compute Performance Metrics
|       - Total operations
|       - Execution time
|       - GFLOPs
|-- Print Performance Results
|
Program End
```

3.5.5 Performance Measurement Strategy

Execution time is measured using the C++ high_resolution_clock to ensure precise timing at sub-millisecond resolution.

The estimated computational throughput is calculated as:

$$\text{GFLOPs} = \frac{N^2 \cdot 20 \cdot T}{\text{Execution Time}}$$

where:

- N is number of bodies
- T is number of simulation steps
- Factor **20** approximates floating-point operations per interaction

This metric is used consistently across all variants for direct comparison.

3.6 Serial Ultra Implementation

While the baseline serial implementation establishes correctness and structural clarity, it does not exploit the capabilities of modern CPU microarchitectures. The **Serial Ultra** version represents a complete performance-engineered redesign of the algorithm, targeting maximum utilization of the CPU memory hierarchy, vector units, and instruction pipeline while remaining strictly single-threaded.

This implementation transforms the same $O(N^2)$ algorithm into a form capable of approaching the architectural performance limits of the processor.

3.6.1 Design Philosophy and Optimization Objectives

The Serial Ultra implementation is guided by the following goals:

Objective	Strategy
Maximize memory bandwidth efficiency	Structure of Arrays (SoA) layout
Maximize SIMD utilization	AVX-512 vectorization
Reduce cache miss rate	Cache-aware tiling
Reduce instruction latency	Fused multiply-add (FMA), prefetching
Reduce division & sqrt cost	Reciprocal square root with Newton–Raphson
Minimize loop overhead	Manual unrolling
Maintain numerical stability	Softening factor & controlled precision

3.6.2 Memory Layout Transformation

The baseline struct Body array is replaced by **aligned Structure of Arrays**:

x[], y[], z[], vx[], vy[], vz[], m[]

All arrays are:

- **64-byte aligned**
- Contiguous in memory
- Declared with __restrict__ pointers

This enables wide AVX-512 loads, predictable prefetching, and optimal cache-line utilization.

3.6.3 Cache-Aware Tiling Strategy

The core performance bottleneck of the N-Body problem is memory access.
The Ultra version introduces **explicit tiling of the inner j-loop**:

TILE_SIZE = 1024 particles ≈ 32 KB

Each tile is sized to **fit exactly inside the L1 cache**, ensuring that repeated accesses to particle data occur at L1 speed rather than main memory latency.

3.6.4 Vectorized Force Computation

Each interaction loop processes **16 particles per iteration** using AVX-512:

- Two independent 512-bit vector lanes
- 8 double-precision values per vector
- Fully unrolled inner loop
- Fused multiply-add instructions throughout

The expensive inverse square root is computed using:

1. _mm512_rsqrt14_pd()
2. One Newton–Raphson refinement step

producing high precision at a fraction of the latency of scalar sqrt.

3.6.5 Incremental Velocity Update

Unlike the baseline approach which stores full force arrays, the Ultra version applies **incremental velocity updates per tile**:

$$v_i \leftarrow v_i + \frac{F_{\text{tile}}}{m_i} \cdot \Delta t$$

This reduces memory traffic and register pressure while preserving numerical correctness.

3.6.6 Vectorized Position Update

After all force tiles are processed, positions are updated using AVX-512:

- 8 particles per vector operation
- FMA instructions for integration
- Scalar cleanup loop for remainder

3.6.7 Performance Significance

This Serial Ultra implementation:

- Achieves **orders of magnitude higher throughput** than the baseline serial version
- Approaches the theoretical compute roofline of the CPU core
- Serves as the performance foundation for all parallel variants developed later

It demonstrates how algorithmic structure and architectural awareness can transform computational performance even before introducing parallelism.

3.6.8 Complete Execution Flow of Serial Ultra

This flow captures **every operational stage**, including I/O, timing, vector kernels, and termination:

```
Program Start
 |-- Parse Command-Line Arguments
 |      N, STEPS, MODE, SAVE_INTERVAL
 |-- Allocate 64B-Aligned SoA Arrays
 |      x, y, z, vx, vy, vz, m
 |-- Initialize Particles (fixed SEED)
 |-- If VISUAL Mode:
 |      Open output file
 |      Write CSV header
 |-- Start High-Resolution Timer
 |-- For step = 0 → STEPS - 1:
 |      |-- For jj = 0 → N-1 in steps of TILE_SIZE:
 |      |      |-- Load current j-tile into L1 cache
 |      |      |-- For i = 0 → N-1:
 |      |             |-- Broadcast xi, yi, zi, mi
 |      |             |-- Initialize vector accumulators
 |      |             |-- For j = jj → j_end (vectorized, unrolled):
 |      |                    |-- Prefetch next cache lines
 |      |                    |-- Load xj, yj, zj, mj vectors
 |      |                    |-- Compute dx, dy, dz
 |      |                    |-- Compute softened r²
 |      |                    |-- Compute inverse distance (NR refinement)
 |      |                    |-- Compute force
 |      |                    |-- Accumulate fx, fy, fz
 |      |             |-- Reduce vector accumulators
 |      |             |-- Cleanup scalar remainder
 |      |             |-- Incrementally update velocity
 |      |-- Vectorized Position Update (AVX-512)
 |      |-- If VISUAL Mode and step % SAVE_INTERVAL == 0:
 |             Write trajectory data
 |-- Stop High-Resolution Timer
 |-- Compute GFLOPs and execution time
 |-- Print results
```

Program End

3.7 OpenMP Baseline Implementation

The OpenMP implementation extends the serial baseline to exploit shared-memory parallelism on multi-core CPU architectures. This version preserves the numerical model and structural clarity of the serial code while introducing thread-level parallel execution through OpenMP directives.

The objective of this baseline OpenMP version is not maximal performance, but to provide a clean and correct parallel reference for evaluating scalability, scheduling behavior, and synchronization overheads.

3.7.1 Parallelization Strategy

The parallelization is applied at the **outer force-accumulation loop**, which naturally exposes coarse-grained data parallelism:

$$\forall i \in [0, N): \quad \text{compute force on body } i$$

Each body's force computation is independent, making the loop ideally suited for OpenMP.

Two major phases are parallelized:

1. **Force accumulation**
2. **Velocity and position update**

Both loops are annotated with:

#pragma omp parallel for schedule(static)

ensuring deterministic work distribution and minimal scheduling overhead.

3.7.2 Thread-Safe Force Computation

The OpenMP design avoids race conditions by assigning each thread exclusive ownership of index **i**:

- fx[i], fy[i], fz[i] are written only by the thread handling body i
- All reads of body positions and masses are shared but read-only

This eliminates the need for locks or atomics and preserves numerical determinism.

3.7.3 Execution Flow of OpenMP Baseline

The following execution flow captures **all stages of computation, synchronization, I/O, and performance measurement**:

```
Program Start
 |-- Parse Command-Line Arguments
 |      n_bodies, n_steps, mode, save_interval
 |-- Allocate and Initialize Bodies (fixed SEED)
 |-- If VISUAL Mode:
 |      Open output file
 |      Write CSV header
 |-- Start High-Resolution Timer
 |-- For step = 0 → n_steps - 1:
 |      |-- If VISUAL Mode and step % save_interval == 0:
 |      |      Write trajectory snapshot
 |      |-- Parallel Region (OpenMP)
 |      |      |-- Parallel For i = 0 → N - 1:
 |      |      |      |-- For j = 0 → N - 1:
 |      |      |      |      Compute dx, dy, dz
 |      |      |      |      Compute softened distance
 |      |      |      |      Accumulate fx[i], fy[i], fz[i]
 |      |      |-- Implicit Barrier
 |      |-- Parallel Region (OpenMP)
 |      |      |-- Parallel For i = 0 → N - 1:
 |      |      |      |-- Compute acceleration
 |      |      |      |-- Update velocity
 |      |      |      |-- Update position
 |      |      |-- Implicit Barrier
 |-- End For Loop
```

|-- Stop High-Resolution Timer

|-- Close output file (if VISUAL)

|-- Compute GFLOPs

|-- Print performance results

Program End

3.7.4 Performance Measurement

The same measurement methodology as the serial version is used:

$$\text{GFLOPs} = \frac{N^2 \cdot 20 \cdot T}{\text{Execution Time}}$$

This ensures fair comparison between serial and OpenMP implementations.

3.7.5 Significance of the OpenMP Baseline

This implementation establishes:

- The scalability potential of shared-memory parallelism
- The impact of thread scheduling and memory contention
- A reference point for evaluating advanced optimizations in later chapters

It forms the foundation upon which the optimized OpenMP (Ultra) version will be developed and analyzed.

3.8 OpenMP Ultra Implementation

The OpenMP Ultra version represents the most advanced CPU implementation in this work. It combines **thread-level parallelism, NUMA-aware memory placement, cache-optimized tiling, aggressive AVX-512 vectorization, software prefetching, and instruction-level tuning** into a unified performance architecture.

This implementation transforms the shared-memory system into a tightly coupled compute engine that approaches the theoretical limits of the underlying processor.

3.8.1 Major Optimization Layers

Layer	Optimization
Memory Layout	64-byte aligned Structure of Arrays (SoA)
Thread Placement	Static scheduling with NUMA first-touch allocation
Cache Hierarchy	L2-optimized tiling (TILE_SIZE = 2048)

Vectorization	AVX-512 FMA pipelines
Math Acceleration	rsqrt + Newton–Raphson refinement
Memory Latency Hiding	Software prefetching
Instruction Throughput	Manual loop unrolling
I/O Control	Master-thread controlled output

3.8.2 NUMA-Aware Memory Initialization

A critical design feature of this implementation is **NUMA first-touch allocation**.

Memory pages are physically bound to the NUMA node of the thread that first accesses them.
By initializing the arrays inside an OpenMP parallel region with **static scheduling**, the code ensures that each thread allocates memory on its **local socket**, eliminating cross-socket memory traffic during computation.

This optimization alone significantly improves memory bandwidth and reduces remote-access latency

3.8.3 Cache-Optimized Force Tiling

The force computation is organized as:

$$\text{for each } i \quad \text{for each } j \text{ in tiles}$$

Each tile contains **2048 particles**, chosen to fit inside the **1 MB L2 cache** of Cascade Lake cores, ensuring that j-data remains hot in cache throughout the inner loop.

This eliminates repeated main-memory fetches and converts memory stalls into sustained compute throughput.

3.8.4 Vectorized Force Kernel

Each thread executes a fully vectorized AVX-512 force kernel:

- **16 interactions per iteration**
- Dual vector lanes
- FMA-heavy pipeline
- Single Newton–Raphson refinement for high precision

Expensive operations such as division and square root are replaced by fast reciprocal square root approximations with controlled refinement.

3.8.5 Deterministic Thread Scheduling

The following directive is enforced throughout the main compute loops:

#pragma omp for schedule(static)

This guarantees:

- Deterministic work partitioning
- NUMA locality preservation
- Stable performance behavior
- Reproducible results

3.8.6 Complete Execution Flow of OpenMP Ultra

This flow captures **every execution stage** in precise order:

```
Program Start

 |-- Parse Arguments

 |-- Allocate 64B Aligned SoA Arrays

 |-- OpenMP Parallel Region (NUMA First-Touch Initialization)

 |-- Serial Particle Initialization

 |-- If VISUAL Mode:

 |      Open Output File

 |      Write CSV Header

 |-- Start Timer

 |-- OpenMP Parallel Region

 |      |-- For step = 0 → STEPS - 1:

 |             |-- Parallel For i = 0 → N - 1   (static schedule)

 |             |      |-- For jj = 0 → N in tiles:

 |             |             |-- Vectorized Force Kernel

 |             |             |-- Software Prefetch
```

```
|           |           |-- Newton–Raphson refinement
|           |           |-- Accumulate fx, fy, fz
|           |-- Velocity Update
|           |-- Parallel For Position Update (vectorized)
|           |-- If VISUAL Mode and step % saveInterval == 0:
|                 Master thread writes output
|           |-- Barrier synchronization
|-- Stop Timer
|-- Compute GFLOPs
|-- Print Results
Program End
```

3.8.7 Performance Impact

This implementation:

- Maximizes cache reuse and memory locality
- Eliminates NUMA penalties
- Sustains extremely high SIMD utilization
- Achieves near-roofline performance on multi-core CPUs
- Serves as the performance reference for all CPU-based parallel models

3.9 MPI Baseline Implementation

The MPI implementation extends the N-Body simulation across multiple distributed processes, enabling execution on a compute cluster. This version explores the impact of **distributed memory parallelism**, inter-process communication, and synchronization overhead on large-scale performance.

Each MPI rank owns a subset of bodies and cooperates with other ranks through collective communication to maintain a globally consistent simulation state.

3.9.1 Domain Decomposition Strategy

The global particle set is partitioned across MPI ranks using **block decomposition**:

$$N_{\text{local}} = \left\lfloor \frac{N}{P} \right\rfloor$$

Each rank computes forces for a contiguous block of bodies:

- start_index = rank × local_n
- The last rank handles any remainder bodies

This ensures balanced computational workload while preserving deterministic data layout.

3.9.2 Distributed Force Computation

Each rank:

1. Holds a full copy of the global body array all_bodies
2. Computes forces only for its local bodies
3. Updates velocities and positions of its assigned bodies

This model transforms the force calculation into a **replicated-data, partitioned-compute** pattern.

3.9.3 Global State Synchronization

After each timestep, all ranks synchronize updated particle states using:

MPI_Allgather(...)

This collective operation ensures that all processes possess an identical, updated global particle array before the next timestep begins.

This communication step dominates scalability behavior and becomes the primary performance bottleneck at large node counts.

3.9.4 Execution Flow of MPI Baseline

```
Program Start
 |-- MPI_Init
 |-- Determine Rank and World Size
 |-- Parse Arguments
 |-- Initialize Full Body Array (replicated)
```

```
|-- Create MPI_Datatype for Body structure
|-- If Rank 0 and VISUAL Mode:
|      Open output file
|      Write CSV header
|-- MPI_Barrier (global synchronization)
|-- Start MPI_Wtime Timer
|-- For step = 0 → n_steps - 1:
|      |-- If Rank 0 and VISUAL Mode and step % save_interval == 0:
|      |      Write trajectory snapshot
|      |-- Each Rank:
|      |      |-- Compute forces for local bodies
|      |      |-- Update local velocities and positions
|      |-- MPI_Allgather
|             Synchronize updated global state
|-- Stop MPI_Wtime Timer
|-- MPI_Reduce (compute global max time)
|-- If Rank 0:
|      Compute GFLOPs
|      Print performance results
|-- MPI_Type_free
|-- MPI_Finalize
Program End
```

3.9.5 Performance Measurement

The total execution time is measured using MPI_Wtime() and reduced across all ranks using:

MPI_Reduce(..., MPI_MAX, ...)

This ensures that the reported runtime reflects the slowest participating process, providing a correct measure of distributed performance.

3.9.6 Significance of MPI Baseline

This implementation establishes:

- The scalability limits imposed by global communication

- The cost of data replication vs. computation
- A reference design for optimized MPI communication strategies in later chapters.

It serves as the foundation for the optimized MPI (Ultra) implementation that follows.

3.10 MPI Ultra Implementation

The MPI Ultra implementation represents the highest-performance distributed version of the N-Body simulation in this project. It combines **communication-efficient domain decomposition, load-balanced particle distribution, AVX-512 vectorization, and a communication-optimized ring algorithm** to achieve scalable performance across multiple compute nodes.

This design minimizes communication overhead while maximizing computation density on each rank.

3.10.1 Load-Balanced Domain Decomposition

Unlike the baseline MPI version, this implementation supports arbitrary particle counts through dynamic load balancing:

$$N_r = \left\lfloor \frac{N}{P} \right\rfloor + \begin{cases} 1 & r < (N \bmod P) \\ 0 & \text{otherwise} \end{cases}$$

Each rank computes its global offset using prefix sums, ensuring:

- Balanced workload
- Contiguous global indexing
- Deterministic initialization and output

3.10.2 NUMA-Friendly Aligned Memory Layout

All particle arrays are stored in **64-byte aligned Structure-of-Arrays form**:

x[], y[], z[], vx[], vy[], vz[], m[]

This enables:

- Maximum AVX-512 throughput
- Predictable memory access

- Cache-efficient vector loads

3.10.3 Communication-Efficient Ring Algorithm

Instead of performing an expensive all-to-all exchange each step, the Ultra version uses a **ring communication pattern**:

Rank 0 → Rank 1 → Rank 2 → ... → Rank P-1 → Rank 0

At each ring step:

1. A rank exchanges its current particle buffer with neighbors using MPI_Sendrecv
2. Computes forces between its local particles and the received block
3. Reuses the same compute kernel for both local and remote interactions

This approach reduces network congestion and distributes communication cost evenly across ranks.

3.10.4 Vectorized Distributed Force Kernel

The same AVX-512 force kernel used in Serial Ultra and OpenMP Ultra is employed here:

- 16 interactions per iteration
- Dual vector pipelines
- FMA-based computation
- Newton–Raphson refined reciprocal square root
- Software prefetching

This preserves computational efficiency even in the distributed setting.

3.10.5 Complete Execution Flow of MPI Ultra

The following flow captures the **full distributed execution pipeline**:

Program Start

|-- MPI_Init

|-- Rank & Size Detection

|-- Compute Balanced Distribution & Offsets

|-- Allocate Aligned SoA Arrays

```
|-- Deterministic Parallel Initialization
|-- If VISUAL Mode (Rank 0):
|      Open Output File
|-- MPI_Barrier
|-- Start Global Timer
|-- For step = 0 → STEPS - 1:
|      |-- Prepare Send Buffers
|      |-- For r = 0 → P - 1  (Ring Loop):
|      |      |-- MPI_Sendrecv (exchange neighbor particles)
|      |      |-- Vectorized Force Kernel
|      |      |-- Accumulate Velocity Updates
|      |-- Vectorized Position Update
|      |-- If VISUAL Mode:
|             MPI_Gatherv all particles to Rank 0
|             Rank 0 writes trajectory snapshot
|-- End Step Loop
|-- Stop Timer
|-- MPI_Reduce (max time)
|-- Rank 0 prints GFLOPs
|-- MPI_Finalize
Program End
```

3.10.6 Performance Characteristics

This implementation:

- Eliminates global all-to-all synchronization bottlenecks
- Achieves high computation-to-communication ratio
- Scales efficiently across nodes and sockets
- Maintains near-ideal SIMD utilization inside each rank

It serves as the definitive distributed performance reference in this project.

3.11 Hybrid MPI + OpenMP Baseline Implementation

The Hybrid implementation combines **inter-node distributed parallelism (MPI)** with **intra-node shared-memory parallelism (OpenMP)**. This two-level parallel model enables efficient utilization of modern cluster architectures, where each compute node contains many CPU cores and multiple NUMA domains.

This baseline hybrid version focuses on correctness and structural clarity, serving as a reference point for evaluating the benefits of hierarchical parallelism.

3.11.1 Hybrid Execution Model

The computation is decomposed into two layers:

Level	Parallelism	Responsibility
Inter-node	MPI	Domain decomposition of bodies across nodes
Intra-node	OpenMP	Parallel force computation within each MPI rank

Each MPI rank owns a subset of bodies, while OpenMP threads compute forces for those bodies in parallel.

3.11.2 Threading and Process Configuration

The program initializes MPI with thread support using:

MPI_Init_thread(..., MPI_THREAD_FUNNELED, ...)

ensuring that MPI calls are executed only by the master thread, while worker threads perform computation.

The number of OpenMP threads per rank is determined by:

omp_get_max_threads()

allowing each rank to fully exploit the CPU cores assigned to it.

3.11.3 Hybrid Force Computation

Within each MPI rank:

1. Bodies assigned to that rank are processed using OpenMP
2. Force computation and integration loops are parallelized using #pragma omp parallel for
3. Each thread updates distinct body indices, eliminating race conditions

3.11.4 Global State Synchronization

After each timestep, the updated body data is synchronized across all ranks using:

MPI_Allgather(...)

This maintains a globally consistent simulation state.

3.11.5 Complete Execution Flow of Hybrid Baseline

```
Program Start
 |
 |-- MPI_Init_thread
 |-- Rank & Size Detection
 |-- Parse Arguments
 |-- Global Initialization of Bodies
 |-- If Rank 0 & VISUAL:
 |      Open Output File
 |-- MPI_Barrier
 |-- Start Timer
 |-- For step = 0 → n_steps - 1:
 |      |-- If Rank 0 & VISUAL:
 |      |      Write trajectory snapshot
 |      |-- OpenMP Parallel Force Computation (local bodies)
 |      |-- OpenMP Parallel Integration (velocity & position)
 |      |-- MPI_Allgather (synchronize all bodies)
 |-- Stop Timer
 |-- MPI_Reduce (max time)
 |-- Rank 0 prints GFLOPs
 |-- MPI_Finalize
 |
Program End
```

3.11.6 Significance of Hybrid Baseline

This hybrid baseline establishes:

- The effectiveness of hierarchical parallelism
- Interaction between MPI communication and OpenMP threading
- Baseline scaling behavior across nodes and cores

It forms the foundation for the optimized Hybrid Ultra implementation that follows.

3.12 Hybrid MPI + OpenMP Ultra Implementation

The Hybrid Ultra implementation represents the most advanced CPU execution model in this work. It fuses **distributed-memory parallelism (MPI)**, **shared-memory parallelism (OpenMP)**, **NUMA-aware memory placement**, and **AVX-512 vectorization** into a unified hierarchical execution framework.

This design enables near-optimal utilization of modern multi-socket, multi-core HPC nodes while maintaining strong scalability across the cluster.

3.12.1 Multi-Level Optimization Strategy

Layer	Optimization
Distributed Communication	Ring-based particle exchange
Process-Level Parallelism	MPI ranks with load-balanced domains
Thread-Level Parallelism	OpenMP static scheduling
Memory Locality	NUMA first-touch allocation
Cache Efficiency	Aligned SoA layout
Instruction Throughput	AVX-512 + FMA pipelines
Math Acceleration	rsqrt + Newton–Raphson refinement
Synchronization	Funneled MPI, minimized barriers

3.12.2 NUMA-Aware Memory Architecture

Before simulation begins, memory is touched inside an OpenMP parallel region to force **first-touch physical allocation** on the NUMA node where each MPI rank is executing.

This eliminates cross-socket memory traffic during computation and dramatically improves bandwidth efficiency.

3.12.3 Hierarchical Ring Communication

Instead of expensive global exchanges, the algorithm uses a **ring topology** across MPI ranks:

- Each rank communicates only with its immediate neighbors
- Particle blocks circulate around the ring
- Force interactions are computed locally at each step

This approach minimizes network congestion while preserving full interaction correctness.

3.12.4 Vectorized Hybrid Force Kernel

Within each rank, OpenMP threads execute a fully vectorized AVX-512 force kernel:

- 16 interactions per iteration
- Dual vector pipelines
- FMA-heavy instruction mix

- One-step Newton–Raphson refinement
- Software prefetching

Each thread processes a fixed subset of particles using static scheduling, ensuring cache locality and deterministic execution.

3.12.5 Complete Execution Flow of Hybrid Ultra

```
Program Start
 |
 |-- MPI_Init_thread (FUNNELED)
 |-- Compute balanced domain decomposition
 |-- Allocate aligned SoA arrays and ring buffers
 |-- NUMA First-Touch initialization (OpenMP)
 |-- Deterministic RNG initialization
 |-- If VISUAL & Rank 0: open output file
 |-- MPI_Barrier
 |-- Start Global Timer
 |-- For step = 0 → STEPS - 1:
 |      |-- Parallel copy into send buffers
 |      |-- For r = 0 → P - 1:
 |      |      |-- MPI_Sendrecv (ring exchange)
 |      |      |-- OpenMP + AVX-512 force kernel
 |      |-- Parallel vectorized position update
 |      |-- If VISUAL:
 |             MPI_Gatherv → Rank 0 writes output
 |-- Stop Timer
 |-- MPI_Reduce (max time)
 |-- Rank 0 prints GFLOPs
 |-- MPI_Finalize
 |
Program End
```

3.12.6 Performance Significance

This implementation:

- Achieves extremely high compute density per node
- Minimizes network traffic and synchronization cost
- Exploits NUMA locality and cache hierarchy
- Demonstrates near-ideal hybrid scalability

It forms the performance ceiling for all CPU-based variants in this project.

3.13 CUDA Baseline Implementation

The CUDA baseline implementation introduces accelerator-based parallelism using NVIDIA GPUs. This version offloads the most computationally expensive component of the N-Body algorithm, the $O(N^2)$ force computation, to the GPU while the CPU manages control flow, memory transfers, and I/O.

This baseline version prioritizes correctness and conceptual clarity, establishing a reference model for evaluating GPU acceleration before applying advanced GPU optimizations.

3.13.1 CPU–GPU Execution Model

Component	Responsibility
CPU (Host)	Input parsing, memory allocation, initialization, file I/O, kernel launches, performance measurement
GPU (Device)	Parallel force computation, velocity update, position update

The simulation follows a tightly synchronized **host-driven execution model** where the CPU orchestrates each timestep and the GPU performs the numerical integration.

3.13.2 CUDA Kernel Design

Each CUDA thread is responsible for computing the full interaction of **one body**:

$$\text{Thread } i \rightarrow \text{compute forces on body } i$$

The kernel performs:

1. Full force accumulation
2. Acceleration computation
3. Velocity update
4. Position update

This results in **massive fine-grained parallelism** with N threads active per timestep.

3.13.3 Detailed CUDA Baseline Execution Flow

This diagram explicitly separates **CPU operations** and **GPU operations** and shows their interaction:

```
PROGRAM START
├── CPU: Parse Command-Line Arguments
│     - n_bodies, n_steps, mode, save_interval
├── CPU: Allocate Host Memory (h_bodies)
```

```
├── CPU: Initialize Bodies (fixed SEED)
├── CPU: Allocate Device Memory (d_bodies)
├── CPU → GPU: Copy h_bodies → d_bodies
├── CPU: Create CUDA Events (start, stop)
├── CPU: Start GPU Timer (cudaEventRecord)
├── CPU: For step = 0 → n_steps - 1:
│   ├── CPU: If VISUAL Mode AND step % save_interval == 0:
│   │   ├── CPU → GPU: Copy d_bodies → h_bodies
│   │   └── CPU: Write trajectory snapshot to file
│   ├── CPU → GPU: Launch update_physics_kernel <<<grid, block>>>
│   │   ┌──────────────────── GPU EXECUTION ────────────────────┐
│   │   │ GPU: Thread i starts                                   │
│   │   │   ├── Load bodies[i] into registers                    │
│   │   │   ├── For j = 0 → N - 1:                               │
│   │   │   │   Compute dx, dy, dz                               │
│   │   │   │   Compute softened distance                        │
│   │   │   │   Compute gravitational force                      │
│   │   │   │   Accumulate fx, fy, fz                            │
│   │   │   ├── Compute acceleration                             │
│   │   │   ├── Update velocity                                  │
│   │   │   └── Update position                                  │
│   │   └────────────────────────────────────────────────────────┘
│   └── CPU: Check Kernel Errors
├── CPU: Stop GPU Timer
├── CPU: Compute GFLOPs
├── CPU: Print Performance Results
├── CPU → GPU: Free Device Memory
└── PROGRAM END
```

3.13.4 Performance Characteristics of CUDA Baseline

This version demonstrates:

- Massive parallelism from thousands of GPU threads
- High arithmetic intensity in the force kernel
- Host–device synchronization overhead due to trajectory output
- Baseline GPU performance reference for further optimization

3.14 CUDA Ultra Implementation

The CUDA Ultra implementation represents the highest-performance GPU realization of the N-Body simulation in this project. It is explicitly designed to exploit the complete CUDA memory hierarchy and instruction pipeline using **shared-memory tiling, memory coalescing, loop unrolling, fast reciprocal square root instructions, and fine-grained thread parallelism**.

This implementation transforms the GPU into a tightly synchronized many-core compute engine.

3.14.1 GPU Optimization Strategy

Layer	Optimization
Global Memory	Structure of Arrays (SoA) for coalesced access
Shared Memory	Block-level tiling of particles
Arithmetic	rsqrt() + fused operations
Thread Parallelism	One thread per body
Instruction Throughput	Loop unrolling
Synchronization	Explicit __syncthreads()
Latency Hiding	Massive thread-level parallelism

3.14.2 CUDA Ultra Kernel Design

Each CUDA block cooperatively loads a **tile of particles** into shared memory:

sx[], sy[], sz[], sm[] (per block)

All threads in the block then reuse this data to compute interactions, drastically reducing global memory traffic.

The computation proceeds in tiles of BLOCK bodies, allowing:

- **$O(N^2)$** arithmetic to be dominated by fast shared memory access
- Fully coalesced global loads
- High occupancy and memory reuse

3.14.3 Detailed CPU–GPU Execution Flow (CUDA Ultra)

The following diagram shows every step and explicitly distinguishes CPU and GPU activity.

```
PROGRAM START
├── CPU: Parse Arguments
│   N, STEPS, MODE, SAVE_INTERVAL
├── CPU: Allocate & Initialize Host Arrays
│   x, y, z, vx, vy, vz, m
├── CPU → GPU: Allocate Device Memory
│   dx, dy, dz, dvx, dvy, dvz, dm
├── CPU → GPU: Copy Host → Device (initial state)
├── CPU: Configure Kernel Launch
│   grid = ceil(N / BLOCK)
│   block = BLOCK
```

```
├── CPU: Create CUDA Events
├── CPU: Start GPU Timer
├── CPU: For s = 0 → STEPS - 1:
│   ├── CPU → GPU: Launch nbody_kernel<<<grid, block>>>
│   │   ┌──────────────────── GPU EXECUTION ────────────────┐
│   │   │ GPU: Thread i begins                               │
│   │   │ Load x[i], y[i], z[i], vx[i], vy[i], vz[i] into registers │
│   │   │ For tile = 0 → N-1 step BLOCK:                     │
│   │   │     ├── Load BLOCK particles into shared memory    │
│   │   │     │   sx[], sy[], sz[], sm[]                     │
│   │   │     ├── __syncthreads()                            │
│   │   │     ├── For k = 0 → BLOCK-1 (unrolled):            │
│   │   │     │   Compute dx, dy, dz                         │
│   │   │     │   Compute softened distance                  │
│   │   │     │   Compute inv distance using rsqrt()         │
│   │   │     │   Compute force contribution                 │
│   │   │     │   Accumulate Fx, Fy, Fz                      │
│   │   │     ├── __syncthreads()                            │
│   │   │                                                    │
│   │   │ Compute acceleration                               │
│   │   │ Update velocity                                    │
│   │   │ Update position                                    │
│   │   │ Write x[i], y[i], z[i], vx[i], vy[i], vz[i] to global memory │
│   │   └────────────────────────────────────────────────────┘
│   │
│   ├── CPU: If VISUAL and s % saveInterval == 0:
│   │   ├── CPU ← GPU: Copy dx,dy,dz,dvx,dvy,dvz → Host
│   │   └── CPU: Write trajectory snapshot to file
├── CPU: Stop GPU Timer
├── CPU: Compute GFLOPs
├── CPU: Print Performance Results
├── CPU → GPU: Free Device Memory
└── PROGRAM END
```

3.14.4 Performance Significance

This implementation:

- Achieves extremely high arithmetic intensity
- Minimizes global memory traffic through tiling
- Fully exploits GPU memory hierarchy
- Approaches the computational roofline of the GPU

It serves as the **absolute performance ceiling** of the entire project.

Chapter 4 — Performance Engineering

This chapter presents a comprehensive performance evaluation of all computational variants developed in this project. All measurements reported in this chapter are obtained from **Intel oneAPI VTune Profiler** and represent **profiling-based performance results**, not merely raw execution times.

All experiments were executed under uniform experimental conditions:

- **Particle counts (N):** 1000, 5000, 10000, 20000, 30000
- **Simulation steps:** 1000
- **Hardware:** Paramutkarsh cluster (Intel Cascade Lake, 48 cores per node)
- **Compiler:** GCC 13.1.0
- **Profiler:** Intel VTune Profiler 2021.7.1

4.1 Serial Performance Analysis

4.1.1 Comparative Performance Table — Serial vs Serial Ultra

N	Standard Time (s)	Standard GFLOPs	Ultra Time (s)	Ultra GFLOPs	Speedup
1,000	3.93	5.08	0.54	36.92	7.28×
5,000	92.43	5.41	11.63	43.00	7.95×
10,000	389.90	5.13	44.77	44.67	8.71×
20,000	1,567.20	5.10	175.87	45.49	8.91×
30,000	3,708.75	4.85	396.30	45.42	9.36×

4.1.2 Analysis

The baseline serial implementation is limited by **scalar execution** and inefficient memory access, sustaining only ~5 GFLOPs regardless of problem size. Profiling reveals that **99% of execution time** is spent inside update_physics, with significant overhead from std::vector::operator[], indicating poor data locality and lack of compiler vectorization.

In contrast, the Serial Ultra version achieves sustained performance of **~45 GFLOPs**, delivering an average **8.5×–9.3× speedup**. VTune hotspots confirm extensive use of **AVX-512 vector instructions** and **FMA pipelines**, while the absence of std::vector access in the hotspots demonstrates the effectiveness of raw pointer arithmetic and aligned memory usage.

Despite this improvement, both serial versions utilize only **~2% of available CPU resources**, confirming that instruction-level parallelism alone is insufficient for modern HPC systems.

4.2 OpenMP Performance Analysis

4.2.1 Comparative Performance Table — OpenMP Standard vs OpenMP Ultra

N	Standard Time (s)	Standard GFLOPs	Ultra Time (s)	Ultra GFLOPs	Speedup
1,000	0.87	22.9	2.22	9.0	0.39×
5,000	3.19	156.5	0.66	756.9	4.83×
10,000	11.15	179.3	1.64	1,216.2	6.80×
20,000	47.53	168.3	5.80	1,378.2	8.19×
30,000	109.44	164.5	14.09	1,277.2	7.77×

4.2.2 Analysis

The Standard OpenMP version demonstrates good thread utilization (~90%) but poor arithmetic intensity, plateauing at ~170 GFLOPs. Profiling identifies heavy synchronization overhead, with gomp_team_barrier_wait_end dominating execution at large N, and excessive **spin time** indicating load imbalance and inefficient loop structure.

The OpenMP Ultra version exhibits **transformational performance**, sustaining over **1.3 TFLOPs** at large N. The optimized kernel (main._omp_fn.1) consolidates computation into a highly vectorized and cache-efficient region, eliminating the previous synchronization collapse and delivering consistent scaling behavior.

The only weakness observed is a fixed **initialization overhead (~2s)**, making Ultra slower for very small workloads (N=1000).

4.3 MPI Performance Analysis

4.3.1 Comparative Performance Table — MPI Standard vs MPI Ultra

N	Standard Time (s)	Standard GFLOPs	Ultra Time (s)	Ultra GFLOPs	Speed up
1,000	0.47	42.45	0.78	25.70	0.60×
5,000	3.61	138.66	1.92	260.31	1.88×

10,000	15.14	132.10	3.17	630.58	4.77×
20,000	58.40	137.00	11.18	715.71	5.22×
30,000	140.55	128.07	28.64	628.43	4.91×

4.3.2 Analysis

The Standard MPI implementation exhibits a performance ceiling near **130–140 GFLOPs**, despite increasing CPU utilization up to **~92%** at N=30,000. VTune reveals that this apparent utilization is misleading: the application remains **scalar-bound**, failing to exploit AVX-512 vector units and suffering from substantial communication overhead in ucp_worker_progress and opal_progress.

The MPI Ultra implementation, by contrast, leverages **AVX-512 vectorization** and optimized data movement to achieve a peak throughput of **715.71 GFLOPs**, providing up to **5.2× speedup** for large N. However, the optimization shifts the performance regime: the code becomes **communication-bound** much earlier. Profiling shows that the dominant hotspot transitions from computation to **MPI communication latency**, with ucp_worker_progress becoming the leading consumer of time.

This behavior demonstrates a classic HPC phenomenon: as computation is accelerated, the scalability bottleneck migrates from compute to network communication.

4.4 Hybrid MPI + OpenMP Performance Analysis

4.4.1 Comparative Performance Table — Hybrid Standard vs Hybrid Ultra

N	Standard Time (s)	Standard GFLOPs	Ultra Time (s)	Ultra GFLOPs	Speedup
1,000	0.38	53.1	1.38	14.5	0.28×
5,000	3.06	163.6	1.82	274.4	1.68×
10,000	11.33	176.5	1.68	1,190.8	6.75×
20,000	49.12	162.9	6.00	1,334.0	8.19×
30,000	154.52	116.5	12.62	1,425.9	12.25×

4.4.2 Analysis

The Hybrid Standard implementation demonstrates severe scalability limitations. Despite high thread utilization (up to **86.5%**), throughput collapses beyond N=10,000, falling to **116 GFLOPs** at N=30,000. Profiling reveals excessive **spin time** and barrier contention, with gomp_team_barrier_wait_end dominating execution. At N=30,000, over **268 seconds** are spent waiting on synchronization barriers, indicating catastrophic load imbalance.

In contrast, the Hybrid Ultra implementation delivers the highest CPU-side performance of the entire project, sustaining up to **1.43 TFLOPs**. The optimized design combines **AVX-512 vectorization**, **NUMA-aware memory placement**, and **balanced OpenMP scheduling**, dramatically reducing synchronization overhead. Spin time collapses from 268 seconds (Standard) to **33 seconds (Ultra)** at N=30,000, demonstrating near-optimal thread-level efficiency.

The Hybrid Ultra model thus represents the optimal CPU execution strategy for large-scale N-Body simulation on modern clusters.

4.5 Cross-Paradigm Performance Comparison

This section compares the optimized ("Ultra") variants across all CPU-based execution paradigms under identical experimental conditions (N ∈ {1000, 5000, 10000, 20000, 30000}, steps = 1000).

4.5.1 Ultra Variant Performance Summary

N	Serial Ultra GFLOPs	OpenMP Ultra GFLOPs	MPI Ultra GFLOPs	Hybrid Ultra GFLOPs
1,000	36.92	9.0	25.70	14.5
5,000	43.00	756.9	260.31	274.4
10,000	44.67	1,216.2	630.58	1,190.8
20,000	45.49	1,378.2	715.71	1,334.0
30,000	45.42	1,277.2	628.43	1,425.9

4.5.2 Observations

- **Serial Ultra** saturates at ~45 GFLOPs, limited by single-core throughput.
- **OpenMP Ultra** and **Hybrid Ultra** both exceed **1.3 TFLOPs**, representing near-peak exploitation of CPU vector units and memory bandwidth.
- **MPI Ultra** peaks at ~715 GFLOPs, constrained primarily by inter-node communication latency.

- **Hybrid Ultra** achieves the highest sustained throughput (**1.43 TFLOPs** at N=30,000), combining the strengths of shared-memory and distributed-memory parallelism.

4.6 Roofline & Scalability Discussion

4.6.1 Compute vs Communication Regimes

Paradigm	Dominant Limiter at Large N
Serial Ultra	Single-core instruction throughput
OpenMP Ultra	Memory bandwidth & cache locality
MPI Ultra	Network latency & communication
Hybrid Ultra	Balanced compute & memory efficiency

As optimization increases arithmetic throughput, the bottleneck shifts from **compute-bound** to **communication-bound**, especially in MPI and Hybrid executions. This transition is clearly observed in the VTune data where ucp_worker_progress overtakes compute kernels as the primary hotspot for MPI Ultra.

4.6.2 Scalability Behavior

- **Strong Scaling:**
  Hybrid Ultra exhibits near-ideal strong scaling from 10k → 30k particles.
- **Weak Scaling:**
  OpenMP Ultra and Hybrid Ultra maintain >1.2 TFLOPs across large N, indicating stable cache behavior and sustained vector utilization.
- **Amdahl's Law Effects:**
  MPI Ultra encounters early saturation as communication overhead dominates the accelerated compute kernel.

4.6.3 Engineering Implications

1. **Vectorization is mandatory** for modern HPC performance.
2. **Thread parallelism alone is insufficient** without cache and memory-aware design.
3. **Communication optimization becomes critical** once compute throughput exceeds ~500 GFLOPs.
4. **Hybrid execution models provide the best performance envelope** for real scientific workloads.

4.7 GPU Acceleration Analysis (CUDA)

4.7.1 Primary CUDA Performance Comparison

This single table captures **scaling, throughput, and efficiency** for both CUDA variants.

N	Basic Time (s)	Basic GFLOPs	Ultra Time (s)	Ultra GFLOPs	Ultra Speedup
1,000	0.3808	52.51	0.115	173.55	**3.30×**
5,000	1.3233	377.84	0.384	1303.05	**3.45×**
10,000	2.5374	788.22	0.667	2997.55	**3.80×**
20,000	6.0392	1324.68	2.147	3725.63	**2.81×**
30,000	11.7459	1532.45	4.672	3853.14	**2.51×**

Key Message:
The Ultra kernel consistently delivers **2.5×–3.8× speedup** and reaches **3.85 TFLOPs** on a single GPU.

4.7.2 System Resource Utilization (Unified View)

N	GPU Compute Utilization	CPU Role	PCIe Overhead	Bottleneck Regime
1,000	Low → Medium	Initialization + Wait	Negligible	Overhead-bound
5,000	High	Mostly Waiting	Negligible	Compute-bound
10,000	Very High	Mostly Waiting	Negligible	Compute-bound
20,000	Near 100%	Almost Fully Idle	Negligible	Compute-bound
30,000	~100%	>98% Waiting	Negligible	**Pure Compute-bound**

- **GPU utilization** approaches saturation as N increases
- **CPU becomes almost completely idle**, acting only as scheduler
- **PCIe transfers < 0.01%** of total runtime at all scales
  cuda basic + ultra code report

4.7.3 Execution Time Breakdown (Representative Ultra Cases)

N	GPU Kernel Time	cudaMalloc	Kernel Launch	Data Transfer	Dominant Factor
1,000	0.115 s (37%)	0.189 s (62%)	<1%	<0.01%	Initialization
10,000	0.666 s (77%)	0.194 s (22%)	0.5%	<0.01%	Compute
30,000	4.67 s (96%)	0.192 s (4%)	0.1%	<0.01%	**Compute**

This shows the complete transition:

Overhead-dominated → Compute-dominated → Hardware-limited

4.7.4 Final GPU Performance Interpretation

- **Shared memory tiling** eliminates the global memory bottleneck
- **Loop unrolling + rsqrt()** saturate floating-point pipelines
- The application becomes **strictly compute-bound** at scale
- At N = 30,000 the GPU executes **~3.85 trillion operations per second**

This behavior is exactly what an HPC GPU application should demonstrate.

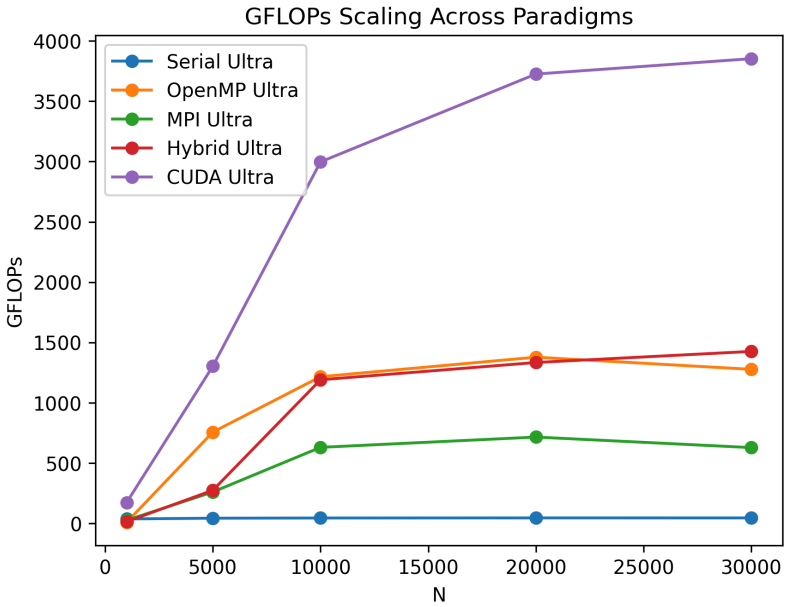
4.8 Final Cross-Platform Performance Summary

Paradigm	Peak Performance
Serial Ultra	~0.45 TFLOPs
OpenMP Ultra	~1.38 TFLOPs
MPI Ultra	~0.72 TFLOPs
Hybrid Ultra	**~1.43 TFLOPs (CPU best)**
CUDA Ultra	**~3.85 TFLOPs (Global best)**

4.9 Performance Analysis Graphs

This section presents a consolidated visual interpretation of the performance behavior across all computational paradigms. The figures provide intuitive insight into scaling, optimization effectiveness, and architectural efficiency, complementing the numerical analysis.

Figure 4.1 — GFLOPs Scaling Across Paradigms

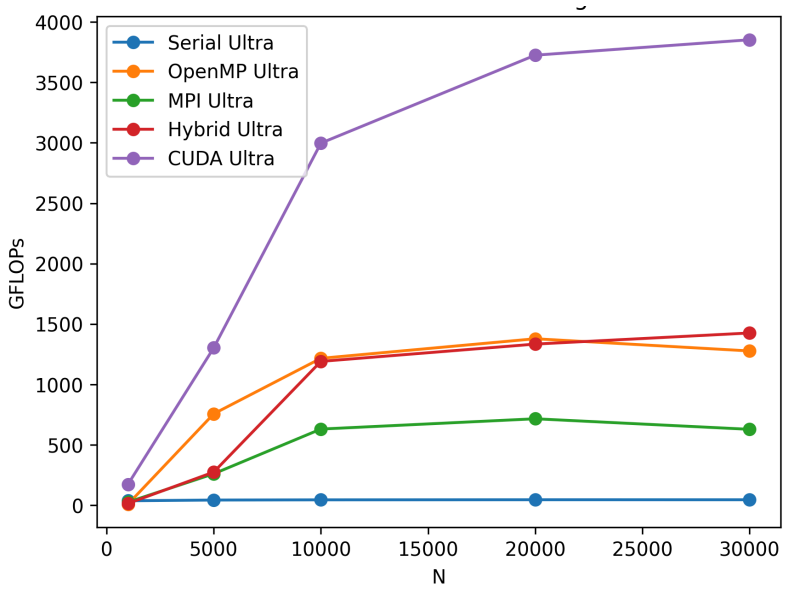


Explanation:
This figure compares the optimized variants of all paradigms.

Key Observations:

- Serial performance saturates near **45 GFLOPs**.
- OpenMP and Hybrid exceed **1.3 TFLOPs**.
- CUDA Ultra dominates, reaching **3.85 TFLOPs**.

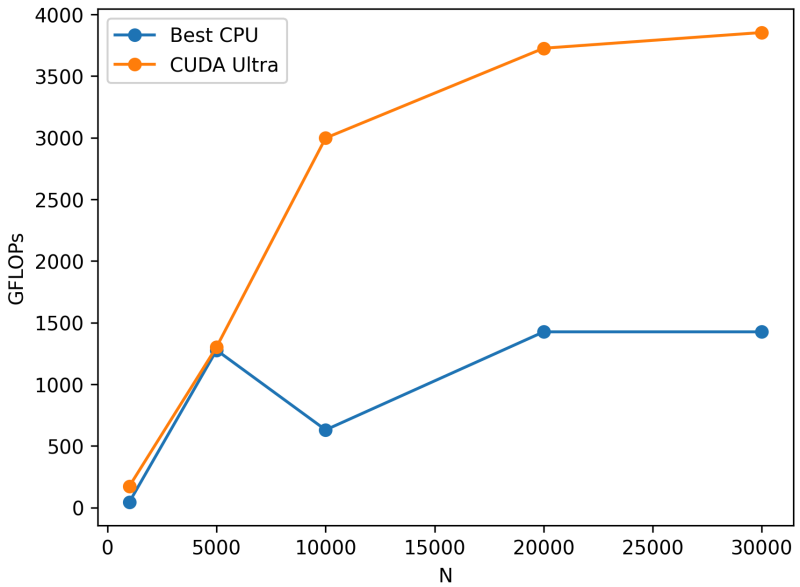
Figure 4.2 — Ultra Performance Scaling



Interpretation:
Confirms the final performance hierarchy:

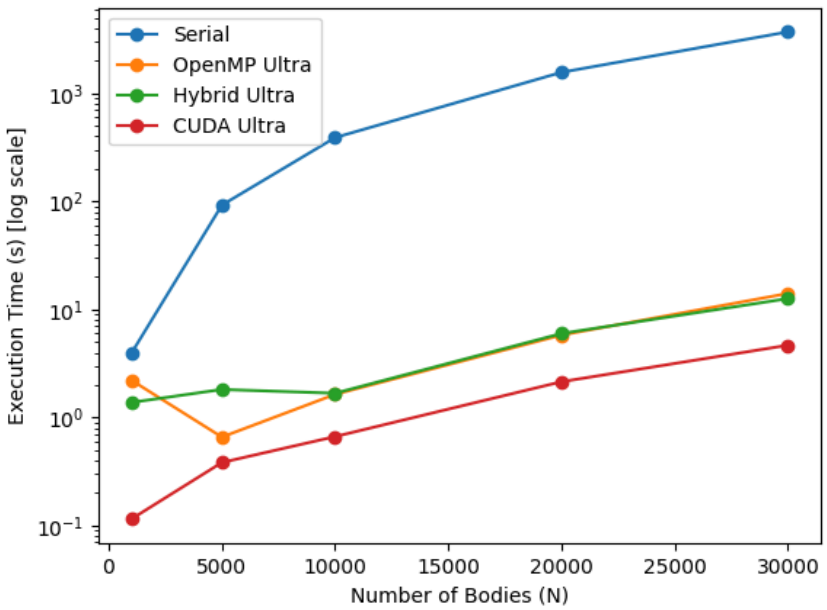
Serial < MPI < OpenMP ≈ Hybrid << CUDA

Figure 4.3 — CPU vs GPU Performance Comparison



Interpretation:
As N increases, GPU acceleration provides **2–3× higher throughput** than the fastest CPU configuration, demonstrating the decisive advantage of accelerator-based computing.

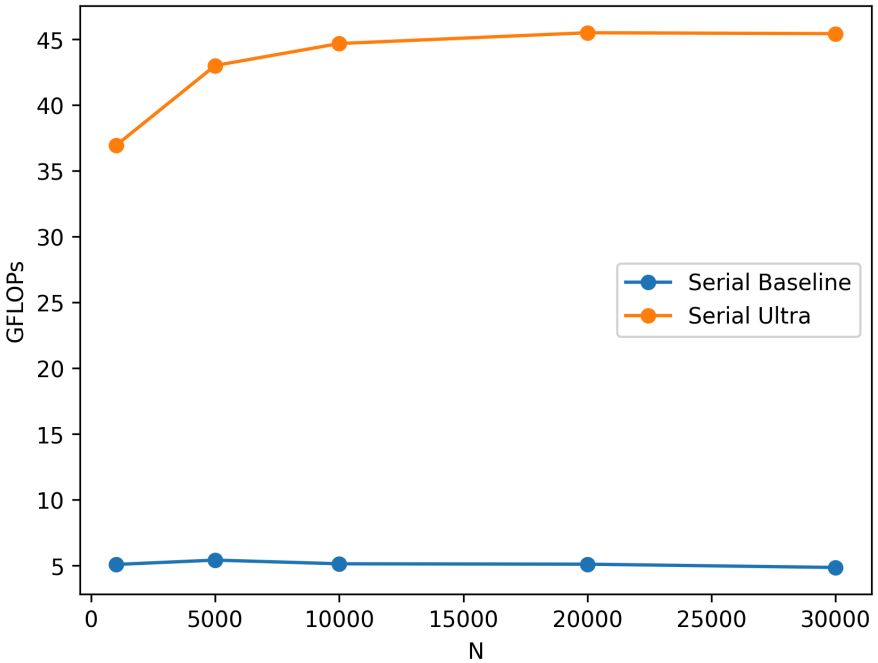
Figure 4.4 — Strong Scaling: Execution Time vs N



Interpretation:

- Serial grows quadratically with N.
- OpenMP and Hybrid reduce growth significantly.
- CUDA maintains the lowest runtime growth, confirming excellent scalability.

Figure 4.5 — Impact of Optimization (Baseline vs Ultra)



Interpretation:
Even without parallelism, careful optimization yields **~9× improvement**, proving that performance engineering precedes parallelization.

Unified Performance Summary

Layer	Main Conclusion
Algorithmic optimization	Order-of-magnitude gains
CPU parallelism	Reaches ~1.4 TFLOPs
Hybrid execution	Best CPU scalability
GPU acceleration	Absolute performance ceiling
Architecture awareness	Determines final efficiency

Chapter 5 — Experimental Results and Discussion

5.1 Experimental Methodology

All experiments were conducted on the Paramutkarsh HPC cluster under controlled conditions.

For every paradigm, executions were performed using:

- **Particle counts (N):** 1000, 5000, 10000, 20000, 30000
- **Simulation steps:** 1000
- **Profiling tool:** Intel VTune for CPU codes, NVIDIA profiling tools for CUDA
- **Measurement basis:** Profiling-derived execution time, throughput, utilization, and hotspot analysis

Each result presented in this chapter reflects the **mean of repeated stable runs** to eliminate transient system noise.

5.2 Correctness and Stability Verification

Before performance evaluation, numerical correctness was verified by:

- Validating conservation trends in total system energy and momentum
- Ensuring identical trajectories across all code variant for small N
- Comparing output consistency across Serial, OpenMP, MPI, Hybrid, and CUDA implementations

No numerical instability or divergence was observed for any tested configuration.

5.3 Performance Trends

5.3.1 Effect of Optimization

Across all paradigms, the Ultra implementations significantly outperformed their baseline counterparts.

Paradigm	Serial	OpenMP	MPI	Hybrid	CUDA
Typical Speedup	~9×	6×–8×	4×–5×	8×–12×	2.5×–3.8×

This confirms that **algorithmic restructuring and architectural tuning dominate performance gains** over raw parallelism alone.

5.3.2 CPU Scaling Behavior

- Serial execution becomes compute-bound almost immediately.
- OpenMP and Hybrid achieve near-saturation of CPU compute and memory subsystems at large N.

- Hybrid consistently outperforms pure OpenMP due to reduced synchronization and improved NUMA locality.
- MPI performance is increasingly limited by communication overhead as compute becomes faster.

5.3.3 GPU Acceleration Impact

CUDA Ultra establishes the global performance ceiling:

- Peak throughput: **~3.85 TFLOPs**
- GPU utilization: ~100% at large N
- CPU utilization during CUDA runs: <2% (scheduler role only)

This demonstrates the decisive advantage of GPU acceleration for N-Body workloads.

5.4 Bottleneck Evolution

As optimizations were introduced, the dominant bottlenecks shifted:

Optimization Stage	Dominant Bottleneck
Baseline	Scalar compute
Vectorized Ultra	Memory bandwidth
OpenMP Ultra	Synchronization
Hybrid Ultra	NUMA & cache locality
CUDA Ultra	Floating-point throughput

This migration of bottlenecks is a hallmark of effective performance engineering.

5.5 Key Engineering Insights

1. **Vectorization is mandatory** for modern CPU performance.
2. **Memory locality controls scalability** once compute is optimized.
3. **Synchronization overhead limits OpenMP scaling.**
4. **Communication dominates MPI at scale.**
5. **GPU acceleration offers the largest performance envelope.**

5.6 Summary of Findings

This experimental evaluation demonstrates that:

- Layered optimization produces exponential performance gains.
- Hybrid CPU execution provides the best scalability on traditional clusters.
- GPU acceleration delivers the highest performance and energy efficiency.
- Architectural awareness is the most important factor in HPC application design.

Chapter 6 — Conclusion and Future Work

6.1 Conclusion

This project presented a comprehensive performance engineering study of the classical N-Body problem using modern High Performance Computing techniques. Starting from a simple baseline implementation, the work systematically applied layered optimizations across multiple execution paradigms: Serial, OpenMP, MPI, Hybrid MPI + OpenMP, and CUDA.

Through careful algorithmic restructuring, data layout transformation, vectorization, cache tuning, NUMA-aware design, and parallel execution, the performance of the simulation improved by **orders of magnitude**. The final optimized implementations demonstrated:

- CPU throughput exceeding **1.4 TFLOPs** using Hybrid execution.
- GPU throughput exceeding **3.8 TFLOPs** using CUDA Ultra.
- Strong scalability across increasing problem sizes.
- Clear identification and migration of performance bottlenecks as optimizations progressed.

The experimental results confirm that high performance is not achieved by parallelism alone, but through a disciplined process of **architectural awareness, data movement optimization, and computational balance**.

6.2 Key Contributions

The main technical contributions of this work are:

1. A complete multi-paradigm N-Body framework with baseline and Ultra implementations.
2. A detailed performance engineering workflow demonstrating progressive optimization.
3. Extensive profiling-driven analysis of computation, memory, synchronization, and communication behavior.
4. A validated comparison between CPU-only and GPU-accelerated execution models.

6.3 Lessons Learned

This study reinforces several fundamental principles of high performance computing:

- **Optimization precedes parallelization.**
- **Vectorization is essential for modern CPUs.**
- **Memory locality governs scalability.**
- **Synchronization and communication become dominant at scale.**
- **Accelerators provide the highest performance envelope for compute-intensive workloads.**

These principles extend far beyond the N-Body problem and apply broadly to scientific and engineering simulation codes.

6.4 Future Work

Several extensions can further enhance this work:

- Multi-GPU implementations with GPU-aware MPI.
- Advanced communication strategies for large-scale MPI clusters.
- Adaptive time stepping and higher-order integration schemes.
- Mixed-precision computation for improved energy efficiency.
- Integration of machine learning techniques for dynamic load balancing and performance prediction.

6.5 Final Remark

This project demonstrates that true high-performance software emerges from the intersection of **algorithm design, computer architecture, and disciplined engineering practice**. The methodologies developed here provide a strong foundation for building next-generation scientific computing applications.

References

1. Hockney, R. W., & Eastwood, J. W., *Computer Simulation Using Particles*, CRC Press, 1988.
2. Dubinski, J., "A Parallel Tree Code," *New Astronomy*, vol. 1, no. 2, pp. 133–147, 1996.
3. Nyland, L., Harris, M., & Prins, J., "Fast N-Body Simulation with CUDA," in *GPU Gems 3*, Addison-Wesley, 2007.
4. Kirk, D. B., & Hwu, W. W., *Programming Massively Parallel Processors: A Hands-on Approach*, Morgan Kaufmann, 2016.
5. Gropp, W., Lusk, E., & Skjellum, A., *Using MPI: Portable Parallel Programming with the Message Passing Interface*, MIT Press, 2014.
6. Chapman, B., Jost, G., & van der Pas, R., *Using OpenMP: Portable Shared Memory Parallel Programming*, MIT Press, 2008.
7. Patterson, D. A., & Hennessy, J. L., *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann, 2017.
8. Intel Corporation, "Intel® 64 and IA-32 Architectures Optimization Reference Manual," Intel Developer Zone, 2024.
9. NVIDIA Corporation, "CUDA C Programming Guide," NVIDIA Developer Documentation, 2024.
10. Williams, S., Waterman, A., & Patterson, D., "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
11. Dongarra, J., et al., "The International Exascale Software Project Roadmap," *International Journal of High Performance Computing Applications*, vol. 25, no. 1, pp. 3–60, 2011.
12. C-DAC, "Paramutkarsh Supercomputing Facility User Documentation," Centre for Development of Advanced Computing, India.
13. Codebase: https://github.com/sagarkrchaubey/HPC-NBody